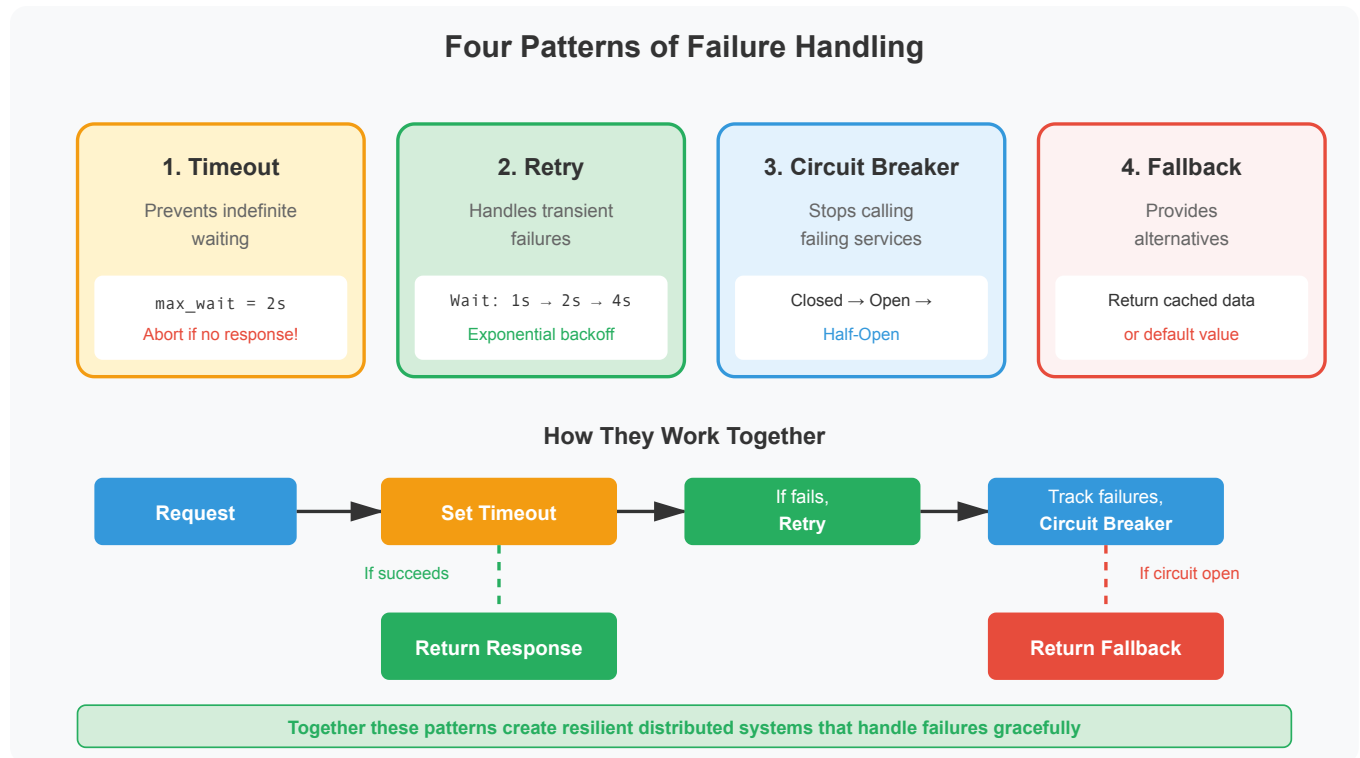


Handling Failures in Synchronous Communication

 systemdesignschool.io/fundamentals/synchronous-communication-failure-handling



Failure Handling in Synchronous Communication

We explored how to implement synchronous communication using REST and gRPC. But networks are unreliable. Services crash. Databases lock up. A production system must handle failures gracefully to prevent cascading outages.

Why Failures Happen

Service B receives 1000 requests per second normally. A bug causes a memory leak. Garbage collection pauses increase. Response time climbs from 50ms to 5 seconds. Service A sends requests and waits. Threads block. New requests queue up. Service A exhausts its thread pool and stops responding. Now Service A's callers time out. The failure spreads upstream.

Networks drop packets. A switch fails and reroutes traffic. Latency spikes from 10ms to 500ms. Requests that normally complete in 100ms now take 600ms. Callers waiting for 200ms timeout and retry. The retry flood overwhelms the network further.

A database deadlock holds a transaction for 30 seconds before aborting. The service waiting for that query blocks. Other requests pile up behind it. Memory pressure builds. The service runs out of resources and crashes.

Preventing Cascading Failures

Four patterns defend against these failure modes. **Timeouts** prevent waiting indefinitely. Set a deadline. If the response does not arrive in time, abort and free resources. A 2-second timeout prevents a thread from blocking for 30 seconds on a deadlocked database.

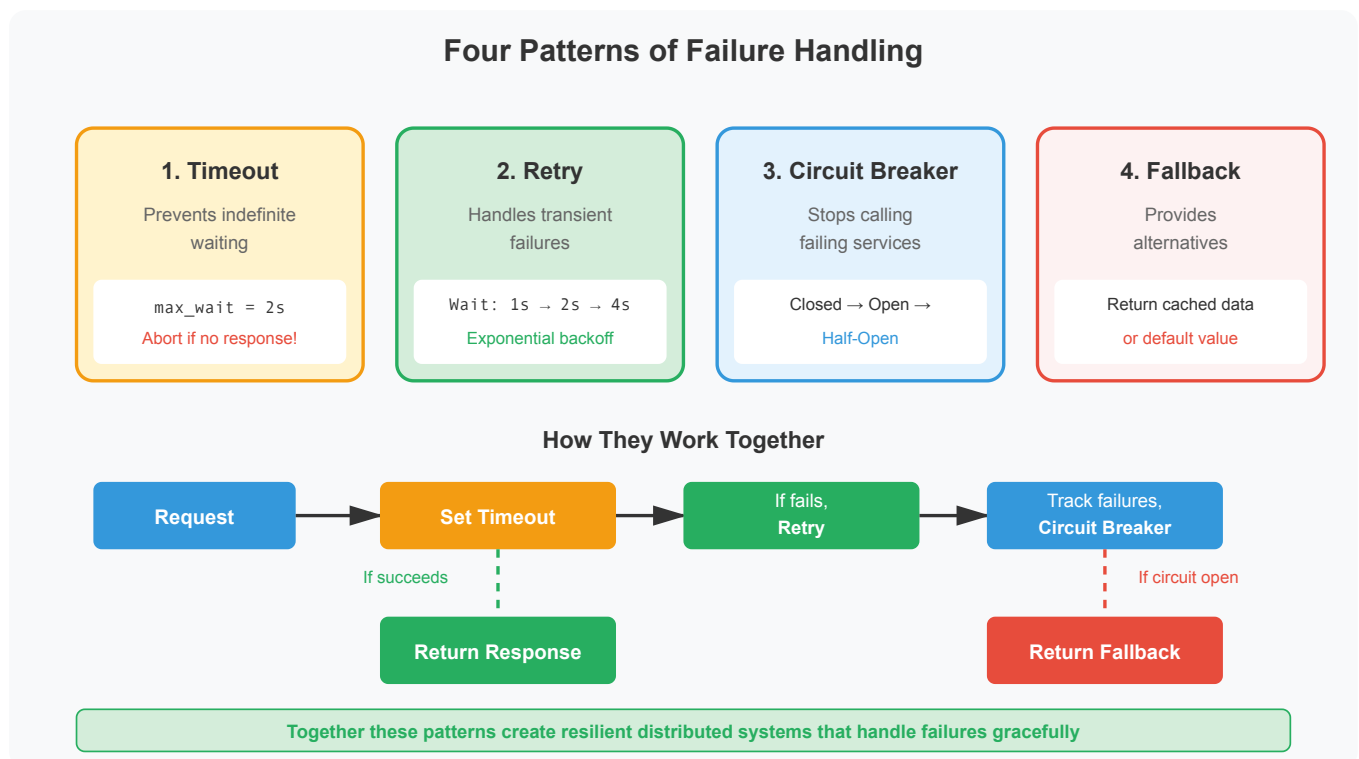
Retries handle transient failures. A network glitch drops a packet. The request fails. Wait briefly and try again. The second attempt succeeds. Exponential backoff prevents retry storms. Wait 1 second, then 2, then 4. Add jitter so clients do not retry simultaneously.

Circuit breakers stop calling failing services. If 50% of requests to Service B fail, open the circuit. Stop sending requests. Return errors immediately. After 30 seconds, try one request. If it succeeds, close the circuit and resume normal traffic. This gives the failing service time to recover without being overwhelmed by continued requests.

Fallbacks provide alternatives when services fail. Return cached data. Show default values. Degrade functionality gracefully. If the recommendations service is down, show popular items instead of personalized suggestions. The user gets a working page rather than an error.

Combining Patterns

Effective systems layer these patterns. A request starts with a timeout to prevent blocking forever. If it fails, retry with exponential backoff. If retries exhaust, the circuit breaker tracks the failure. After enough failures, the circuit opens and stops wasting resources on a service that is clearly down. Meanwhile, the application serves a fallback response so users see degraded functionality rather than errors.



The following sections explore each pattern in detail. [Timeouts](#) cover setting appropriate deadlines. [Retries](#) explain exponential backoff and jitter. [Circuit breakers](#) show state transitions and threshold tuning. [Fallbacks](#) demonstrate graceful degradation strategies.